Machine Learning

Beginner Reading Material

A simple, example-driven guide for students who are new to machine learning.

- **Module I:** Fundamentals of Machine Learning
- **Module II:** Supervised Learning - Classification
- **Module III:** Supervised Learning - Regression

Simple idea: Every topic follows the same pattern: a plain-language idea, a small everyday example, and short Python code that you can run yourself. Read a section, then try the code before moving on.

Contents

Module I: Fundamentals of Machine Learning 2

1.1 Introduction to Machine Learning . . . 2

1.2 Types of Machine Learning . . . 2

1.3 Applications and Use Cases . . . 3

1.4 The Machine Learning Pipeline . . . 3

1.5 Python Libraries for ML . . . 4

1.6 Data Preprocessing Techniques . . . 5

Module II: Supervised Learning - Classification 7

2.1 K-Nearest Neighbors (KNN) . . . 7

2.2 Decision Trees . . . 8

2.3 Naive Bayes and CountVectorizer . . . 9

2.4 Support Vector Machines (SVM) . . . 10

2.5 Ensemble Methods . . . 10

2.6 Performance Metrics and the Confusion Matrix . . . 11

2.7 Handling Imbalanced Datasets . . . 12

2.8 Multi-class Classification Strategies . . . 13

2.9 Bias, Variance, Overfitting and Underfitting . . . 13

2.10 Cross Validation . . . 14

2.11 Hyperparameter Tuning . . . 15

2.12 Feature Engineering . . . 16

Module III: Supervised Learning - Regression 17

3.1 Simple Linear Regression . . . 17

3.2 Multiple Linear Regression . . . 18

3.3 Polynomial Regression . . . 19

3.4 Gradient Descent . . . 19

3.5 Assumptions of Linear Regression . . . 21

3.6 Regularisation: Ridge, Lasso and Elastic Net . . . 22

3.7 Measuring Regression Models . . . 23

Quick Revision and Practice Questions 24

Cheat sheet: which algorithm or tool when? . . . 24

Practice questions . . . 24

Module I: Fundamentals of Machine Learning

In this module you will learn what machine learning is, the different kinds of learning, how a real ML project flows from start to finish, and how to prepare data using Python. Everything is explained with small, everyday examples.

1.1 Introduction to Machine Learning

Machine Learning (ML) is teaching a computer to learn patterns from **data** instead of giving it a long list of hand-written rules.

Traditional programming: you write the rules. Rules + Data = Answers.
Machine learning: the computer finds the rules. Data + Answers = Rules (a *model*).

> **Simple idea:** Think of teaching a child to recognise mangoes. You do not describe every rule about shape and colour. You show many mangoes and say 'this is a mango'. After enough examples the child can recognise a new mango. ML works the same way.

Key terms every beginner must know

Term	Meaning	Example (house price data)
Dataset	Collection of examples (a table)	1,000 rows of houses
Feature (input, X)	A column used to make the prediction	Area, bedrooms, city
Label / Target (y)	The answer we want to predict	Price of the house
Sample / Row	One example	One house
Model	The learned pattern/formula	Price = 3000 x area + ...
Training	The model learning from data	Fitting on past sales
Prediction	Model's answer for new data	Price of a new house

1.2 Types of Machine Learning

1. Supervised Learning (learning with answers)

The data has both inputs **and** correct answers (labels). The model learns the link between them. It is like studying with an answer key. There are two kinds:

- **Classification**: the answer is a category. Example: is this email Spam or Not Spam? Will a student Pass or Fail?
- **Regression**: the answer is a number. Example: predict a house price, tomorrow's temperature, or exam marks.

2. Unsupervised Learning (learning without answers)

The data has **no labels**. The model finds hidden groups or structure on its own. Example: a shop groups its customers into 'budget buyers', 'regular buyers' and 'big spenders' without anyone telling it the groups in advance (this is **clustering**).

3. Reinforcement Learning (learning by trial and error)

An **agent** takes actions in an **environment** and gets **rewards** or **penalties**. It learns which actions give the most reward over time. Example: a robot learning to walk, or a program learning to play chess or a video game.

	Supervised	Unsupervised	Reinforcement
Data	Inputs + labels	Inputs only	Rewards from actions
Goal	Predict the label	Find hidden patterns	Maximise total reward
Example	Spam detection, price prediction	Customer grouping	Game-playing AI, robots

This course material focuses on supervised learning (Modules II and III).

1.3 Applications and Use Cases

Area	What ML does	Type
Email	Filters spam	Classification
Banking	Detects fraudulent transactions	Classification
Healthcare	Predicts whether a tumour is benign or malignant	Classification
Real estate	Predicts house prices	Regression
Retail / OTT	Recommends products or movies	Mostly unsupervised / hybrid
Marketing	Groups customers into segments	Unsupervised
Transport	Self-driving cars, route planning	Supervised + Reinforcement
Phones	Face unlock, voice assistants, auto-correct	Supervised

1.4 The Machine Learning Pipeline

Every ML project follows roughly the same steps. We will use one running example: **predicting whether a student will pass an exam** using study hours and attendance.

Step	What happens	Student-pass example
1. Data Collection	Gather data from files, databases, forms, sensors, websites	Collect hours studied, attendance and result for 500 past students
2. Preprocessing	Clean the data: fix missing values, scale numbers, encode text	Fill missing attendance, scale hours and attendance
3. Model Building	Split into train/test sets, choose an algorithm, train it	Train a Decision Tree on 80% of students
4. Evaluation	Test on unseen data and measure accuracy and other metrics	Check how many of the other 20% were predicted correctly
5. Deployment	Put the model into a real app or website so people can use it	A web form: enter hours and attendance, get Pass/Fail

Remember: Data preparation usually takes most of the time in real projects (often more than half). A simple model on clean data beats a fancy model on messy data.

1.5 Python Libraries for ML

NumPy: fast numbers and arrays

NumPy gives you **arrays**, which are fast lists of numbers that support maths on the whole list at once.

```
import numpy as np

marks = np.array([35, 45, 50, 60, 70])
print(marks.mean())       # 52.0  (average)
print(marks + 5)          # [40 50 55 65 75]  adds 5 to every value
print(marks.shape)        # (5,)
```

Pandas: working with tables

Pandas gives you the **DataFrame**, which is like an Excel sheet inside Python.

```
import pandas as pd

df = pd.DataFrame({
    "hours":  [1, 2, 3, 4, 5],
    "attend": [60, 70, 80, 85, 95],
    "passed": [0, 0, 1, 1, 1]
})
print(df.head())          # first rows
print(df.describe())      # count, mean, min, max ...
print(df[df["hours"] > 2])  # filter rows
df.isnull().sum()         # count missing values per column
```

Scikit-learn: the ML toolbox

Scikit-learn (`sklearn`) contains ready-made ML algorithms, preprocessing tools and metrics. Almost every model follows the **same three steps**:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X = df[["hours", "attend"]]      # features
y = df["passed"]                 # label

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier()   # 1. create the model
model.fit(X_train, y_train)        # 2. train (learn) on training data
pred = model.predict(X_test)       # 3. predict on new data
```

> **Remember:** `fit()` = learn, `predict()` = answer. You will use this pattern for every algorithm in this course.

1.6 Data Preprocessing Techniques

Real data is messy. Models need clean, numeric data. Preprocessing fixes this.

A. Handling Missing Values

Sometimes a cell is empty (NaN). Options:

Method	How	When to use
Drop rows	Remove rows that have missing values	Very few rows are missing
Mean / Median	Fill with the column's average or middle value	Numeric columns. Median is better when there are outliers
Mode	Fill with the most frequent value	Category columns (e.g. city)
Constant	Fill with 0 or 'Unknown'	When 'missing' itself has a meaning

```
from sklearn.impute import SimpleImputer

imp = SimpleImputer(strategy="mean")      # or "median", "most_frequent"
X_filled = imp.fit_transform(X)

# Or in pandas:
df["attend"] = df["attend"].fillna(df["attend"].mean())
df = df.dropna()                          # drop rows with any NaN
```

B. Feature Scaling

Suppose *age* ranges 20 to 60 but *salary* ranges 20,000 to 2,00,000. Salary numbers are so big that some algorithms (KNN, SVM, gradient descent) will treat it as far more important. Scaling puts all features on a similar range.

Method	Formula	Result
Min-Max Scaling (Normalisation)	(x - min) / (max - min)	Values between 0 and 1
Standardisation (Z-score)	(x - mean) / standard deviation	Mean 0 and standard deviation 1

Example: Ages 20, 30, 50. Min = 20, max = 50. Min-Max gives (20-20)/30 = 0, (30-20)/30 = 0.33, (50-20)/30 = 1.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

scaler = StandardScaler()                       # or MinMaxScaler()
X_train_s = scaler.fit_transform(X_train)       # learn mean/std from TRAIN only
X_test_s  = scaler.transform(X_test)            # apply the same to TEST
```

Remember: Always **fit the scaler on training data only**, then only `transform` the test data. Fitting on test data leaks information (called data leakage). Tree-based models (Decision Tree, Random Forest) do not need scaling.

C. Encoding Categorical Data

Models understand numbers, not words. We convert categories to numbers.

Method	Idea	Use when
Label Encoding	Small -> 0, Medium -> 1, Large -> 2	Categories have a natural order (ordinal)
One-Hot Encoding	One new 0/1 column per category	No order exists (city, colour)

One-hot example: a *city* column with Delhi, Pune and Hyderabad becomes three columns.

city	city_Delhi	city_Pune	city_Hyderabad
Delhi	1	0	0
Pune	0	1	0
Hyderabad	0	0	1

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df2 = pd.DataFrame({"city": ["Delhi", "Pune", "Hyderabad"]})
pd.get_dummies(df2, columns=["city"])          # one-hot encoding

sizes = pd.Series(["Small", "Large", "Medium"])
sizes.map({"Small": 0, "Medium": 1, "Large": 2})   # ordered (label) encoding
```

Remember: Module I summary. ML learns patterns from data. Supervised = labels (classification or regression), Unsupervised = no labels, Reinforcement = rewards. The pipeline is Collect, Preprocess, Build, Evaluate, Deploy. Clean the data first: fix missing values, scale numbers, encode categories.

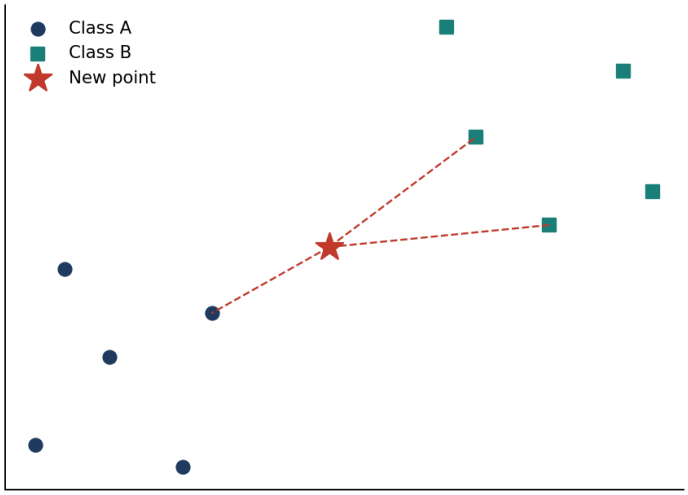
Module II: Supervised Learning - Classification

Classification means predicting a **category**: Spam or Not Spam, Pass or Fail, Disease or Healthy, or one of several flowers. This module covers the most popular classification algorithms, how to measure whether a model is good, and how to improve it.

2.1 K-Nearest Neighbors (KNN)

> **Simple idea:** 'You are like the people around you.' To classify a new point, look at its **K nearest neighbours** and let them vote.

Steps: (1) choose K, (2) measure the distance from the new point to every training point (usually straight-line *Euclidean* distance), (3) pick the K closest, (4) the majority class among them is the prediction.



With K = 3, the 3 nearest neighbours of the new point decide its class.

- **Small K** (like 1): very sensitive to noise, can overfit.
- **Large K**: very smooth, can underfit. Choose an odd K to avoid ties (3, 5, 7).
- **Scale your features** first. KNN is based on distance, so big-valued columns dominate.
- KNN is a 'lazy learner': it does no real training, it just stores the data. Prediction on large data is slow.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

sc = StandardScaler()
X_tr, X_te = sc.fit_transform(X_tr), sc.transform(X_te)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_tr, y_tr)
print("Accuracy:", knn.score(X_te, y_te))
```

2.2 Decision Trees

> **Simple idea:** A decision tree is a flow-chart of yes/no questions, like the game '20 Questions'. Each question splits the data into purer groups.

Example: will a student pass? Question 1: 'Attendance above 75%?' If yes, ask 'Studied more than 2 hours?' If no, predict Fail. At the end of each branch is a **leaf** that gives the answer.

How does the tree choose the best question?

It picks the split that makes the groups as **pure** as possible (mostly one class). Purity is measured by:

- **Gini impurity** = 1 - (p1 squared + p2 squared + ...), where p is the fraction of each class.
- **Entropy** = measure of disorder. High when classes are mixed, 0 when pure.

> **Example:** A group with 5 Pass and 5 Fail: Gini = 1 - (0.5 x 0.5 + 0.5 x 0.5) = 0.5 (most mixed). A group with 10 Pass and 0 Fail: Gini = 1 - (1 + 0) = 0 (perfectly pure). The tree prefers splits that lower Gini the most.

Pros: easy to understand and explain, no scaling needed, handles numbers and categories. **Cons:** a deep tree memorises the training data (overfitting). Control it with `max_depth`, `min_samples_split` and `min_samples_leaf`.

```
from sklearn.tree import DecisionTreeClassifier, export_text

tree = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=42)
tree.fit(X_tr, y_tr)
print("Accuracy:", tree.score(X_te, y_te))
print(export_text(tree))      # prints the tree as readable rules
```

2.3 Naive Bayes and CountVectorizer

Naive Bayes

Naive Bayes uses **probability** (Bayes' theorem) to decide the most likely class. It is called 'naive' because it assumes every feature is independent of the others, which is rarely fully true, but it still works very well, especially for **text** such as spam filtering and sentiment analysis.

Bayes' theorem: P(class | words) = P(words | class) x P(class) / P(words)

> **Example:** In your mailbox 40% of emails are spam, so P(spam) = 0.4. The word 'free' appears in 50% of spam emails and in 5% of normal emails. For an email containing 'free':
> Spam score = 0.5 x 0.4 = 0.20. Normal score = 0.05 x 0.6 = 0.03.
> P(spam | free) = 0.20 / (0.20 + 0.03) = **about 0.87**. So the email is probably spam.

Variant	Use for
MultinomialNB	Word counts (text classification). The most common choice
GaussianNB	Continuous numeric features (assumes a bell curve)
BernoulliNB	Yes/No features (word present or not)

CountVectorizer: turning text into numbers

Models cannot read words. **CountVectorizer** builds a vocabulary of all unique words and turns each document into a row of **word counts** (the 'bag of words' idea: the order of words is ignored, only counts matter).

Example: three short messages.

Message	at	free	meeting	money	noon	now	offer
free money now	0	1	0	1	0	1	0
meeting at noon	1	0	1	0	1	0	0
free offer now	0	1	0	0	0	1	1

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

texts  = ["win free money now", "free offer now", "meeting at noon",
          "lunch at noon", "free cash prize", "project meeting today"]
labels = [1, 1, 0, 0, 1, 0]            # 1 = spam, 0 = not spam

cv = CountVectorizer()                  # lowercase, split words, count
X = cv.fit_transform(texts)             # learn vocabulary and convert
print(cv.get_feature_names_out())       # the vocabulary

nb = MultinomialNB()
nb.fit(X, labels)

new = cv.transform(["free money offer"])   # use transform (not fit) on new text
print(nb.predict(new))                      # [1]  -> spam
```

Remember: Useful options: `stop_words='english'` removes common words like 'the' and 'is'. `ngram_range=(1,2)` also counts two-word phrases. `TfidfVectorizer` is a similar tool that reduces the weight of very common words.

2.4 Support Vector Machines (SVM)

Simple idea: SVM draws the **best dividing line** between two classes: the one with the **widest gap (margin)** on both sides.

- **Hyperplane**: the dividing line (a line in 2D, a plane in 3D, and so on).
- **Margin**: the empty space between the line and the closest points of each class. SVM maximises it.
- **Support vectors**: the closest points that 'support' (define) the margin. Other points do not matter.
- **Kernel**: a trick to separate data that no straight line can split. `linear` for straight boundaries, `rbf` for curved boundaries (the default).
- **C**: strictness. Large C = fewer mistakes allowed (risk of overfitting). Small C = a wider, more forgiving margin.

SVM works well on small and medium datasets, but **scaling is essential**, and it is slow on very large datasets.

```
from sklearn.svm import SVC

svm = SVC(kernel="rbf", C=1.0)
svm.fit(X_tr, y_tr)               # X_tr was scaled earlier
print("Accuracy:", svm.score(X_te, y_te))
```

2.5 Ensemble Methods

An **ensemble** combines many models to get a better answer than any single model, like asking a panel of experts instead of one person.

Random Forest (many trees, vote together)

Build many decision trees. Each tree is trained on a **random sample of the rows** and uses **random subsets of features**. For a new example every tree votes and the majority wins. This idea is called **bagging**. Because the trees make different mistakes, the mistakes cancel out, which greatly reduces overfitting.

Gradient Boosting (trees that fix each other's mistakes)

Build trees **one after another**. Each new small tree focuses on the examples the previous trees got wrong. The trees are added together, step by step, to form a strong model. This idea is called **boosting**.

	Random Forest	Gradient Boosting
How trees are built	Independently (in parallel)	One by one (sequentially)
Main benefit	Reduces variance (less overfitting)	Reduces bias (more accurate)
Tuning	Easy, works well by default	More careful (learning rate, trees, depth)
Speed	Fast to train	Slower to train

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_tr, y_tr)
print("Random Forest:", rf.score(X_te, y_te))
print(rf.feature_importances_)         # which features mattered most

gb = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,
                                max_depth=3, random_state=42)
gb.fit(X_tr, y_tr)
print("Gradient Boosting:", gb.score(X_te, y_te))
```

2.6 Performance Metrics and the Confusion Matrix

How do we know if a classifier is good? We test it on data it has never seen and compare its predictions with the truth. The **confusion matrix** is a table that shows exactly where the model is right and wrong.

Example: a spam filter is tested on 100 emails (spam = the 'positive' class).

	Predicted: Spam	Predicted: Not Spam
Actually Spam	TP = 30 (True Positive)	FN = 20 (False Negative)
Actually Not Spam	FP = 10 (False Positive)	TN = 40 (True Negative)

- **TP**: spam correctly caught. **TN**: normal mail correctly left alone.
- **FP**: normal mail wrongly sent to spam (a Type I error). **FN**: spam that slipped through (a Type II error).

Metric	Formula	Our example	Question it answers
Accuracy	(TP + TN) / total	(30 + 40) / 100 = 0.70	Overall, how often is the model right?
Precision	TP / (TP + FP)	30 / 40 = 0.75	Of the emails we called spam, how many really were?
Recall (Sensitivity)	TP / (TP + FN)	30 / 50 = 0.60	Of all real spam, how much did we catch?
F1-Score	2 x P x R / (P + R)	2 x 0.75 x 0.60 / 1.35 = 0.67	A single balance of precision and recall

Remember: Which metric matters? If a false alarm is costly (a missed job-offer email going to spam), focus on **precision**. If missing a case is dangerous (cancer screening, fraud), focus on **recall**. If you need both, use **F1**.

ROC curve and AUC

Most classifiers actually output a **probability** and we choose a **threshold** (usually 0.5) to turn it into a class. The **ROC curve** plots the True Positive Rate (recall) against the False Positive Rate for **every possible threshold**. The **AUC** (Area Under the Curve) summarises it in one number:

- AUC = 1.0: perfect model. AUC = 0.5: no better than random guessing. Below 0.5: worse than random.
- In simple words, AUC is the probability that the model gives a higher score to a random positive example than to a random negative example.

```
from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score)

pred = rf.predict(X_te)
print(confusion_matrix(y_te, pred))
print(classification_report(y_te, pred))      # precision, recall, F1 per class

# Binary problem example (needs probabilities, not just 0/1 predictions):
# proba = model.predict_proba(X_te)[:, 1]
# print(roc_auc_score(y_te, proba))
```

2.7 Handling Imbalanced Datasets

A dataset is **imbalanced** when one class is far more common than the other. In a bank, 99 out of 100 transactions are genuine and only 1 is fraud.

Example: A lazy model that always says 'Not Fraud' is **99% accurate** but catches **0%** of frauds (recall = 0). This is the **accuracy paradox**. On imbalanced data, never trust accuracy alone.

Ways to handle it

Technique	Idea
Use better metrics	Look at precision, recall, F1 and PR-AUC, not accuracy
Class weights	Tell the model that mistakes on the rare class cost more (class_weight='balanced')
Oversampling	Add more rare-class examples: copy them (random oversampling) or create new similar ones (SMOTE)
Undersampling	Remove some examples of the common class (you lose data)
Stratified splitting	Keep the same class ratio in train and test sets
Threshold tuning	Lower the 0.5 threshold so the model flags more positives

```
# Option 1: class weights (simplest)
rf = RandomForestClassifier(class_weight="balanced", random_state=42)

# Option 2: SMOTE (pip install imbalanced-learn)
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE(random_state=42).fit_resample(X_train, y_train)

# Always keep the stratified ratio when splitting:
train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
```

> **Remember:** Resample **only the training data**. The test set must stay untouched so it reflects the real world.

2.8 Multi-class Classification Strategies

Binary classification has 2 classes (spam / not spam). **Multi-class** has 3 or more (Iris flowers: setosa, versicolor, virginica). Some algorithms (KNN, Decision Tree, Random Forest, Naive Bayes) handle many classes naturally. Others (SVM, Logistic Regression) are built for two classes and use a strategy:

Strategy	How it works	Number of models for N classes
One-vs-Rest (OvR / OvA)	One model per class: 'this class vs all the others'. Choose the most confident model.	N
One-vs-One (OvO)	One model for every pair of classes. Majority vote decides.	N x (N-1) / 2

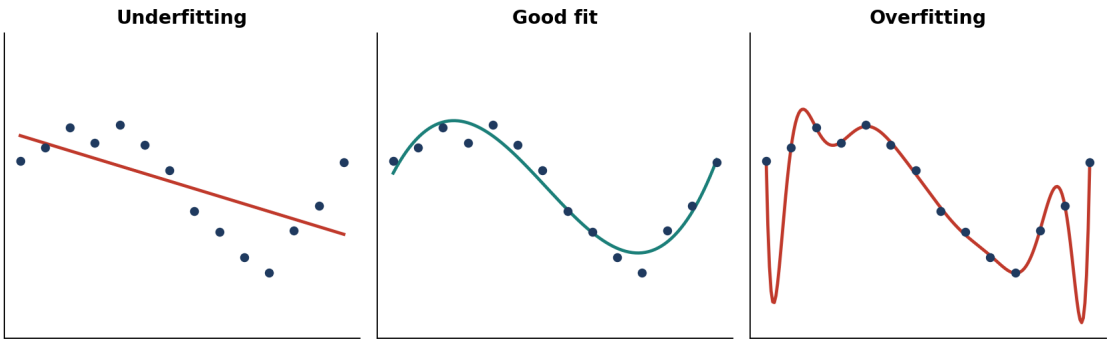
Example: for 3 flower types, OvR trains 3 models; OvO trains 3 models (A-B, A-C, B-C). For 10 classes, OvR needs 10 models but OvO needs 45.

```
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier
from sklearn.svm import SVC

ovr = OneVsRestClassifier(SVC()).fit(X_tr, y_tr)
ovo = OneVsOneClassifier(SVC()).fit(X_tr, y_tr)

# For multi-class metrics, choose how to average across classes:
from sklearn.metrics import f1_score
f1_score(y_te, ovr.predict(X_te), average="macro")   # every class counts equally
```

2.9 Bias, Variance, Overfitting and Underfitting



Same data, three models: too simple, just right, too complex.

- **Underfitting (high bias)**: the model is too simple and misses the real pattern. It does badly on **both** training and test data. Like a student who barely studied.
- **Overfitting (high variance)**: the model is too complex and memorises noise. Very good on training data, poor on new data. Like a student who memorised last year's question paper but cannot solve new questions.
- **Good fit**: captures the main pattern and does well on both.

	Training score	Test score	Fix
Underfitting	Low	Low	Use a more complex model, add features, train longer, lower regularisation
Good fit	High	High (close to training)	-
Overfitting	Very high	Much lower	More data, simpler model, regularisation, cross-validation, ensembles, limit tree depth

Remember: Bias-variance trade-off: making a model more complex lowers bias but raises variance. Making it simpler does the opposite. The goal is the sweet spot in the middle where total error on new data is lowest.

2.10 Cross Validation

A single train/test split can be lucky or unlucky depending on which rows land in the test set. **Cross-validation** repeats the test several times on different parts of the data and averages the results, giving a more trustworthy score.

K-Fold Cross Validation

Split the data into K equal parts (**folds**). Train K times. Each time, use a different fold as the test set and the remaining K-1 folds for training. Average the K scores.

Round (K = 5)	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
1	TEST	train	train	train	train
2	train	TEST	train	train	train
3	train	train	TEST	train	train
4	train	train	train	TEST	train
5	train	train	train	train	TEST

Stratified K-Fold

Normal K-Fold may put most of a rare class in one fold. **Stratified K-Fold** keeps the **same class percentage in every fold**, so use it for classification, especially with imbalanced data.

```
from sklearn.model_selection import cross_val_score, KFold, StratifiedKFold

model = RandomForestClassifier(random_state=42)

kf  = KFold(n_splits=5, shuffle=True, random_state=42)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(model, X, y, cv=skf, scoring="accuracy")
print(scores)                          # 5 scores, one per fold
print("Mean:", scores.mean(), " Std:", scores.std())
```

2.11 Hyperparameter Tuning

Parameters are learned by the model from data (like split points in a tree). **Hyperparameters** are settings **you choose before training**, such as K in KNN, `max_depth` in a tree, or C in SVM. Tuning means finding the best combination.

	Grid Search CV	Random Search CV
Idea	Try **every** combination in a grid you define	Try a fixed number of **randomly chosen** combinations
Pros	Thorough, will not miss a combination in the grid	Much faster when there are many options
Cons	Gets very slow as the grid grows	May miss the exact best combination
Use when	Few hyperparameters / small ranges	Many hyperparameters / big ranges

Both use cross-validation internally to score each combination. If the grid has 3 x 3 = 9 combinations and cv = 5, Grid Search trains 45 models.

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

params = {"n_estimators": [50, 100, 200],
          "max_depth": [3, 5, 10]}

grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    params, cv=5, scoring="f1_macro")
grid.fit(X_tr, y_tr)
print(grid.best_params_)              # best combination
print(grid.best_score_)               # its cross-validated score
best_model = grid.best_estimator_     # ready-trained best model

rand = RandomizedSearchCV(RandomForestClassifier(random_state=42),
                          params, n_iter=5, cv=5, random_state=42)
rand.fit(X_tr, y_tr)
```

2.12 Feature Engineering

Feature engineering means creating or improving input columns so the model can learn more easily. Good features often help more than a fancier algorithm.

Technique	Example
Create new features	From height and weight create BMI = weight / height squared
Extract from dates	From '2026-10-12' get day of week, month, is_weekend
Binning	Convert age into groups: child, young, adult, senior
Combine features	price_per_sqft = price / area
Encode and scale	One-hot encode cities, scale numbers (Module I)
Text to numbers	CountVectorizer / TF-IDF
Feature selection	Remove useless or duplicate columns to reduce noise and speed up training

```
df["bmi"] = df["weight"] / (df["height"] ** 2)
df["month"] = pd.to_datetime(df["date"]).dt.month
df["age_group"] = pd.cut(df["age"], bins=[0, 12, 25, 60, 120],
                         labels=["child", "young", "adult", "senior"])
```

Remember: Module II summary. Classifiers: KNN (vote of neighbours), Decision Tree (yes/no questions), Naive Bayes (probability, great for text with CountVectorizer), SVM (widest margin), Random Forest and Gradient Boosting (teams of trees). Judge them with the confusion matrix, precision, recall, F1 and AUC, not accuracy alone. Fight overfitting with cross-validation, and find the best settings with Grid or Random Search.

Module III: Supervised Learning - Regression

Regression means predicting a **number**: a house price, tomorrow's temperature, a student's marks, a company's sales. The most basic and important regression model is **linear regression**, and many other ideas in ML build on it.

3.1 Simple Linear Regression

> **Simple idea:** Simple linear regression draws the **best-fitting straight line** through the data points, using **one** input to predict the output.

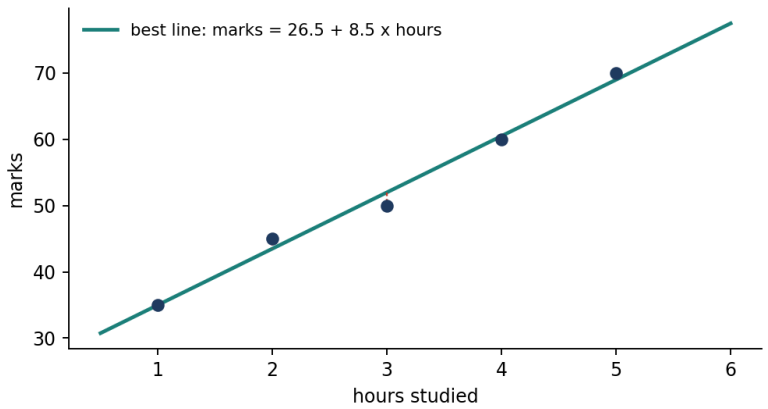
Equation of the line: $y = b0 + b1\ x$

- **y**: the value we predict (marks). **x**: the input (hours studied).
- **b0 (intercept)**: the predicted y when x = 0 (where the line crosses the y-axis).
- **b1 (slope)**: how much y changes when x increases by 1.

Example: five students, hours studied vs marks.

Hours (x)	1	2	3	4	5
Marks (y)	35	45	50	60	70

The line that fits best turns out to be **marks = 26.5 + 8.5 x hours**. Meaning: a student who studies 0 hours is predicted 26.5 marks, and every extra hour adds about 8.5 marks. For 6 hours the prediction is 26.5 + 8.5 x 6 = **77.5 marks**.



The dashed red lines are the errors (residuals) between each point and the line.

What does 'best-fitting' mean?

For each point, the **residual** is the gap between the actual value and the line's prediction. Linear regression chooses b0 and b1 that make the **sum of squared residuals as small as possible** (the **least squares** method). Squaring stops positive and negative errors from cancelling and punishes big errors more. The average of the squared errors is called the **Mean Squared Error (MSE)**, and it is the 'cost' that we try to minimise.

Formulas (for reference): b1 = sum((x - mean_x)(y - mean_y)) / sum((x - mean_x) squared), and b0 = mean_y - b1 x mean_x. For our data: mean_x = 3, mean_y = 52, the top part is 85, the bottom part is 10, so b1 = 8.5 and b0 = 52 - 8.5 x 3 = 26.5.

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4], [5]])      # X must be 2-D (rows x columns)
y = np.array([35, 45, 50, 60, 70])

model = LinearRegression()
model.fit(X, y)

print(model.intercept_)       # 26.5   (b0)
print(model.coef_)            # [8.5]  (b1)
print(model.predict([[6]]))   # [77.5]
```

3.2 Multiple Linear Regression

Real outcomes depend on **more than one** input. **Multiple linear regression** uses several features:

y = b0 + b1 x1 + b2 x2 + b3 x3 + ...

> **Example:** House price = 5,00,000 + 3,000 x (area in sq ft) + 2,00,000 x (bedrooms) - 10,000 x (age of house in years). Each coefficient says how much the price changes when that feature increases by 1 **while the other features stay the same**. So a bedroom adds 2,00,000 and each extra year of age lowers the price by 10,000.

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

data = pd.DataFrame({
    "area":     [800, 1000, 1200, 1500, 1800, 2000],
    "bedrooms": [1, 2, 2, 3, 3, 4],
    "age":      [15, 10, 8, 5, 3, 1],
    "price":    [30, 42, 50, 68, 80, 95]          # in lakhs
})
X = data[["area", "bedrooms", "age"]]
y = data["price"]

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.33, random_state=1)
model = LinearRegression().fit(X_tr, y_tr)

print(dict(zip(X.columns, model.coef_)))     # weight of each feature
print(model.intercept_)
print(model.predict([[1400, 3, 6]]))         # price of a new house
```

Tip: if two features carry nearly the same information (for example, 'area in sq ft' and 'area in sq metres'), the coefficients become unstable. This problem is called **multicollinearity** (see section 3.5).

3.3 Polynomial Regression

Sometimes the relationship is **curved**, not a straight line. For example, the braking distance of a car grows faster than its speed. A straight line would fit poorly (underfitting). **Polynomial regression** adds powers of x so the model can bend:

$$y = b0 + b1\,x + b2\,x^2 + b3\,x^3 + \ldots$$

The number of the highest power is called the **degree**. Degree 1 is a straight line, degree 2 is a curve with one bend (parabola), and so on. Technically it is still 'linear' regression, because we simply create new columns (x squared, x cubed) and fit a linear model on them.

- Degree too low: underfitting (cannot follow the curve).
- Degree too high: overfitting (wiggles through every point, as in the 'Overfitting' picture in Module II).
- Choose the degree using cross-validation or by checking the test score.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
import numpy as np

speed = np.array([[10], [20], [30], [40], [50], [60]])
brake = np.array([5, 12, 24, 40, 62, 90])          # braking distance

model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
model.fit(speed, brake)
print(model.predict([[45]]))
```

3.4 Gradient Descent

The formula for linear regression can be solved directly, but for large and complex models (neural networks, for example) we use **gradient descent**, a method that finds the best values by taking many small steps downhill.

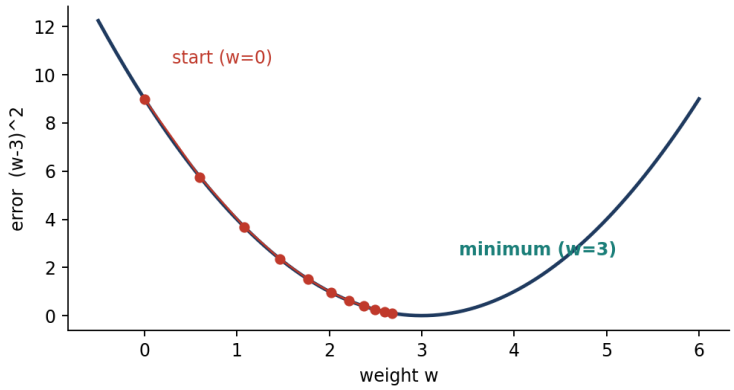
> **Simple idea:** Imagine standing on a foggy hill and wanting to reach the valley. You cannot see far, but you can feel which direction slopes downward. So you take a small step in that direction, feel the slope again, and repeat until the ground becomes flat. The valley is the **lowest error**.

Update rule: new weight = old weight - learning rate x gradient (slope of the error)

- **Gradient**: the slope of the error at the current weight. It tells us which way is uphill.
- **Learning rate** (alpha or lr): the size of each step. We move **against** the gradient to go downhill.
- Repeat until the error stops decreasing (convergence).

Tiny worked example. Minimise error = (w - 3) squared, whose slope is 2(w - 3). Start at w = 0 with learning rate 0.1.

Step	Current w	Slope 2(w - 3)	New w = w - 0.1 x slope
1	0.00	-6.00	0.60
2	0.60	-4.80	1.08
3	1.08	-3.84	1.46
...	...	...	keeps moving towards 3



Each red dot is one step. Steps get smaller as the slope flattens near the minimum.

Choosing the learning rate

Learning rate	What happens
Too small	Very slow. Needs thousands of steps
Just right	Steady progress to the minimum
Too large	Overshoots the valley, bounces around or even diverges (error grows)

Three flavours

Type	Uses how much data per step	Notes
Batch GD	All the training data	Stable but slow on big data
Stochastic GD (SGD)	One example at a time	Fast and noisy
Mini-batch GD	A small group (e.g. 32 examples)	Best balance, most used in practice

Gradient descent for linear regression from scratch on the study-hours data. It finds the same answer as before (b1 = 8.5, b0 = 26.5):

```
import numpy as np

x = np.array([1, 2, 3, 4, 5])
y = np.array([35, 45, 50, 60, 70])

w, b = 0.0, 0.0            # start with any guess
lr = 0.01                  # learning rate

for step in range(5000):
    pred  = w * x + b
    error = pred - y
    dw = (2 / len(x)) * np.sum(error * x)     # slope of MSE w.r.t. w
    db = (2 / len(x)) * np.sum(error)         # slope of MSE w.r.t. b
    w -= lr * dw                              # step downhill
    b -= lr * db

print(round(w, 2), round(b, 2))               # 8.5  26.5
```

Remember: Scale your features before using gradient descent. If one feature has huge values and another has small values, the descent zig-zags and converges slowly. In scikit-learn, `SGDRegressor` performs gradient-descent-based regression.

3.5 Assumptions of Linear Regression

Linear regression gives reliable results only when the data roughly follows these assumptions:

Assumption	Simple meaning	How to check / fix
1. Linearity	The relationship between each feature and the target is roughly a straight line	Scatter plot. If curved, use polynomial regression or transform the feature (e.g. log)
2. Independence of errors	One observation's error does not influence another's (e.g. no time-order pattern)	Plot residuals in time order. Use time-series methods if there is a pattern
3. Homoscedasticity	Errors have a constant spread at all prediction levels	Residuals vs predictions plot should look like an even band, not a funnel. Try log-transforming y
4. Normality of errors	Residuals are roughly bell-shaped (matters mainly for confidence intervals)	Histogram or Q-Q plot of residuals
5. No multicollinearity	Features are not strongly correlated with each other	Correlation matrix or VIF (above 5 to 10 is a warning). Drop or combine features, or use Ridge

Remember: Also watch out for **outliers**. A few extreme points can pull the line away from the real trend, because squared errors punish big gaps heavily.

```
residuals = y_te - model.predict(X_te)

import matplotlib.pyplot as plt
plt.scatter(model.predict(X_te), residuals)    # should look like random noise
plt.axhline(0, color="red")
plt.xlabel("Predicted"); plt.ylabel("Residual")
plt.show()

print(X.corr())                                 # check feature correlations
```

3.6 Regularisation: Ridge, Lasso and Elastic Net

When a model has many features, it can fit the training data too perfectly, with very large coefficients, and overfit. **Regularisation** adds a **penalty for large coefficients** to the cost, which forces the model to stay simple:

New cost = MSE + penalty on the size of the coefficients

> **Simple idea:** A teacher says: 'You can write as many points as you like in your answer, but every extra complicated point costs you marks.' Students then include only what truly matters. Regularisation does the same to the model's coefficients.

	Ridge (L2)	Lasso (L1)	Elastic Net
Penalty	alpha x sum of squared coefficients	alpha x sum of absolute coefficients	Mix of L1 and L2
Effect	Shrinks all coefficients towards 0 but rarely to exactly 0	Can push some coefficients to **exactly 0**	Shrinks and removes, in a balanced way
Feature selection?	No	Yes (automatic)	Yes (partly)
Best when	Many features that all matter a little, correlated features	Many useless features, you want a simple model	Many correlated features and you also want selection
sklearn	Ridge(alpha=1.0)	Lasso(alpha=0.1)	ElasticNet(alpha=0.1, l1_ratio=0.5)

- **alpha** controls the strength. Alpha = 0 means no regularisation (ordinary linear regression). Larger alpha means a stronger penalty and a simpler model. Too large leads to underfitting.
- **l1_ratio** (Elastic Net): 1 behaves like Lasso, 0 behaves like Ridge, 0.5 is an equal mix.
- **Always scale features first**, because the penalty depends on the size of the coefficients.

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

ridge = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
lasso = make_pipeline(StandardScaler(), Lasso(alpha=0.1))
enet  = make_pipeline(StandardScaler(), ElasticNet(alpha=0.1, l1_ratio=0.5))

for name, m in [("Ridge", ridge), ("Lasso", lasso), ("ElasticNet", enet)]:
    m.fit(X_tr, y_tr)
    print(name, "test R2:", m.score(X_te, y_te))

print(lasso[-1].coef_)       # some values may be exactly 0 (dropped features)

# Choose alpha automatically with cross-validation:
from sklearn.linear_model import RidgeCV
RidgeCV(alphas=[0.01, 0.1, 1, 10, 100]).fit(X_tr, y_tr)
```

3.7 Measuring Regression Models

For regression we measure how **far** the predictions are from the actual numbers.

Example: actual marks are 50, 60, 70 and the model predicted 52, 57, 73. The errors are 2, -3 and 3.

Metric	Meaning	Our example
MAE (Mean Absolute Error)	Average size of the errors, ignoring sign	(2 + 3 + 3) / 3 = 2.67
MSE (Mean Squared Error)	Average of squared errors (punishes big errors)	(4 + 9 + 9) / 3 = 7.33
RMSE	Square root of MSE, in the same unit as y	square root of 7.33 = 2.71 marks
R-squared (R2)	Fraction of the variation in y that the model explains. 1 is perfect, 0 is no better than predicting the average	0.89 (the model explains 89%)

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_true = [50, 60, 70]
y_pred = [52, 57, 73]
print(mean_absolute_error(y_true, y_pred))             # 2.67
print(mean_squared_error(y_true, y_pred))              # 7.33
print(np.sqrt(mean_squared_error(y_true, y_pred)))     # 2.71
print(r2_score(y_true, y_pred))                        # 0.89
```

Remember: Module III summary. Linear regression fits a line (y = b0 + b1 x) by minimising squared errors. Multiple regression uses many features, polynomial regression adds curves, and gradient descent finds the best weights by repeated small downhill steps. Check the assumptions. Use Ridge (L2), Lasso (L1) or Elastic Net to prevent overfitting.

Quick Revision and Practice Questions

Cheat sheet: which algorithm or tool when?

Situation	Choice
Predict a category	Classification: KNN, Decision Tree, Naive Bayes, SVM, Random Forest, Gradient Boosting
Predict a number	Regression: Linear, Polynomial, Ridge, Lasso, Elastic Net
Text data	CountVectorizer + Naive Bayes
Need an easy-to-explain model	Decision Tree
Want strong results with little tuning	Random Forest
Features have very different ranges	Scale them (needed for KNN, SVM, gradient descent, regularisation)
Rare positive class (fraud, disease)	Use recall, precision, F1, class weights, SMOTE, stratified split
Model great on train, poor on test	Overfitting: simplify, regularise, add data, cross-validate
Model poor on both	Underfitting: add features or complexity
Find best hyperparameters	GridSearchCV or RandomizedSearchCV

Practice questions

Short answer

1. Differentiate between supervised, unsupervised and reinforcement learning with one example each.

2. List the five stages of the machine learning pipeline.

3. Why do we fit a scaler only on the training data? What is data leakage?

4. When would you use one-hot encoding instead of label encoding?

5. Explain how KNN classifies a new point. What happens if K is too small or too large?

6. What is Gini impurity? Calculate it for a node with 6 Pass and 2 Fail students.

7. Why is Naive Bayes called 'naive'? What does CountVectorizer do?

8. What are support vectors and margin in SVM?

9. Compare Random Forest and Gradient Boosting.

10. A model has TP = 40, FP = 10, FN = 20, TN = 130. Calculate accuracy, precision, recall and F1.

11. Why is accuracy misleading on imbalanced data? Name three ways to handle imbalance.

12. Explain overfitting and underfitting. How can each be fixed?

13. Why is Stratified K-Fold preferred for classification? How many models does GridSearchCV train if the grid has 6 combinations and cv = 5?

14. Write the equation of simple linear regression and explain intercept and slope.

15. Explain gradient descent in your own words. What happens if the learning rate is too large?

16. List the assumptions of linear regression.

17. Differentiate Ridge, Lasso and Elastic Net. Which can do feature selection?

Answers to the numerical questions

- **Gini (6 Pass, 2 Fail):** p = 0.75 and 0.25. Gini = 1 - (0.5625 + 0.0625) = 0.375.
- **Metrics (TP 40, FP 10, FN 20, TN 130):** total = 200. Accuracy = 170/200 = 0.85. Precision = 40/50 = 0.80. Recall = 40/60 = 0.67. F1 = 2 x 0.80 x 0.67 / (0.80 + 0.67) = about 0.73.
- **GridSearchCV models:** 6 combinations x 5 folds = 30 model fits (plus one final refit on all training data).

Small coding exercises

- Load the Iris dataset, scale it, and compare the accuracy of KNN (K = 3, 5, 7), a Decision Tree and an SVM.
- Build a spam classifier with CountVectorizer and MultinomialNB on 10 sentences that you write yourself.
- Fit LinearRegression on your own marks-vs-hours data, then compare with Ridge and Lasso.
- Run GridSearchCV on a Random Forest with at least two hyperparameters and report the best settings.